

Documentation technique

Fichier `--main--.py`

Application de statistiques sportives

Projet 1A — 2026

Ce document décrit l'ensemble des fonctions et de la classe présentes dans le fichier `--main--.py`, qui constitue le point d'entrée de l'application et implémente l'intégralité de l'interface en ligne de commande.

Table des matières

Architecture générale

Le fichier `__main__.py` est organisé en trois parties distinctes :

1. **Fonctions utilitaires de niveau module** — fonctions indépendantes pour l'affichage et la lecture du terminal.
2. **Classe CLI** — contient toute la logique de navigation entre les pages de l'application.
3. **Point d'entrée `main()`** — instancie la CLI et lance la boucle principale.

La navigation repose sur un **pattern état-machine** : chaque *page* est une fonction callable retournant soit une autre fonction-page (avancer), soit la chaîne `"back"` (reculer), `"quit"` (quitter) ou `None` (rester sur la page courante). Deux piles `_back` et `_fwd` maintiennent l'historique, à l'image d'un navigateur web.

Fonction `_make_sports_config()`

Type : fonction de niveau module.

Construit et retourne le dictionnaire de configuration complet de l'application. Tous les modules d'analyse (`src.Analysis.*`) sont importés **à l'intérieur** de cette fonction, de façon à retarder leur chargement au premier appel (les données CSV ne sont lues qu'à ce moment).

Chaque clé du dictionnaire est le nom affiché d'un sport. La valeur associée décrit sa structure :

- **collectif** (bool) — détermine si le sport propose une vue Équipe et une vue Joueurs.
- **genre** (bool) — détermine si une sélection Hommes / Femmes est proposée en premier.
- **equipe / joueurs / stats** — listes de descripteurs de statistiques. Chaque descripteur contient :
 - **label** — texte affiché dans le menu.
 - **fn** — la fonction d'analyse à appeler.
 - **inputs** — liste de champs libres à saisir (vide dans la plupart des cas).
 - **selector** (optionnel) — décrit comment générer la liste de choix. Deux variantes existent :
 - **options_fn** — liste simple, affichée en bloc (ex : liste d'équipes).
 - **options_df_fn** — DataFrame avec pagination et filtres (ex : liste de joueurs).

Fonctions utilitaires d'affichage

Ces six fonctions sont de simples helpers de terminal, sans état.

`_titre(texte)`

Affiche un titre encadré par une ligne de tirets (). Utilisé en en-tête de chaque page.

`_option(num, texte)`

Affiche une ligne de menu au format `[N] texte`.

`_nav(peut_reculer, peut_avancer)`

Affiche la barre de navigation contextuelle en bas de page : `[-] Retour`, `[+] Suivant`, `[q] Quitter`. Les options retour et suivant sont conditionnées par les paramètres booléens.

`_lire(prompt)`

Lit une saisie utilisateur via `input()` et retourne la chaîne nettoyée (`strip()`). Le prompt par défaut est `"Votre choix :"`.

`_afficher_resultat(result)`

Affiche un résultat : si `result` est un `DataFrame` non vide, il est converti en texte via `to_string(index=False)`. Si le `DataFrame` est vide, affiche `"(aucun résultat)"`. Si la fonction appelée a déjà imprimé directement (ex : `bracket`), cette fonction ne fait rien.

`_extract_year(val)`

Extrait une année depuis des formats hétérogènes :

- `datetime` ou `Timestamp` → attribut `.year`.
- `int` ou `float` supérieur à 19 000 000 → format `YYYYMMDD`, division par 10 000.
- `int` ou `float` inférieur → déjà une année.
- `str` → lecture des 4 premiers caractères.

Cette fonction est utilisée par le système de filtres sur les années de naissance, où les données proviennent de sources aux formats variés.

Classe CLI

La classe CLI encapsule l'intégralité de la navigation de l'application. Elle maintient :

- `_back` — pile des pages visitées (navigation arrière).
- `_fwd` — pile des pages en avant (navigation avant).
- `_filters` — dictionnaire des filtres actifs (`type_sport`, `categorie`, `type_competition`).
- `_config` — la configuration complète des sports, chargée au démarrage.

Méthode `__init__()`

Initialise les deux piles de navigation à des listes vides et les filtres à `None`. La configuration `_config` est initialisée à `None` et sera remplie lors de l'appel à `run()`.

Méthode `run()`

Point de départ de la boucle principale. Charge la configuration via `_make_sports_config()`, puis entre dans une boucle infinie qui :

1. Appelle la page courante (`current()`).

2. Interprète le résultat :

- `"quit"` → affiche le message d'au revoir et sort.
- `"back"` → dépile `_back`, empile dans `_fwd`, change de page.
- `"forward"` → dépile `_fwd`, empile dans `_back`, change de page.
- Un *callable* → empile la page courante dans `_back`, vide `_fwd`, change de page.
- `None` → reste sur la page courante (réaffichage).

Méthode `_sports_visibles()`

Appelle `src.Model.sport.filtrer()` avec les trois filtres actifs pour obtenir la liste des sports correspondants, puis la croise avec les clés de `_config` pour ne retourner que les sports effectivement disponibles dans l'application.

Méthode `_filtres_str()`

Retourne une chaîne lisible des filtres actifs pour l'affichage dans le menu principal (ex : `"Collectif, E-sport"`). Retourne `"Aucun"` si aucun filtre n'est activé.

Pages de navigation de la classe CLI ---

Méthode `_page_accueil()`

Page racine de l'application. Affiche les filtres actifs et la liste numérotée des sports visibles. L'option `[0]` mène à la page des filtres, les autres options mènent aux pages des sports. Retourne une fonction-page ou une chaîne de contrôle.

Méthode `_page_filtres()`

Page de configuration des filtres. Propose trois catégories de filtres combinables :

- **Format** : Collectif / Individuel.
- **Discipline** : Sport / E-sport.
- **Compétition** : Par points / Éliminatoire / Mixte.

Chaque catégorie dispose d'une option de réinitialisation. Les filtres actifs sont indiqués par le symbole `[]` en face de l'option. La modification d'un filtre retourne `None` (la page se réaffiche immédiatement avec l'état mis à jour).

Factories de pages de la classe CLI ---

Ces méthodes ne sont pas elles-mêmes des pages : elles **créent et retournent** des fonctions-pages (pattern *factory*). Ce choix permet de paramétrer dynamiquement le comportement de chaque page sans dupliquer de code.

Méthode `_make_page_sport(sport)`

Lit la configuration du sport (`collectif, genre`) et délègue vers la factory appropriée :

- Avec genre → `_make_page_genre`.
- Sans genre, collectif → `_make_page_cat`.
- Individuel → `_make_page_stats`.

Méthode `_make_page_genre(titre, config)`

Crée une page proposant le choix Hommes / Femmes. Selon la valeur de `collectif`, délègue ensuite vers `_make_page_cat` (sport collectif avec genre, ex : Volleyball) ou `_make_page_stats` (sport individuel avec genre, ex : Tennis).

Méthode `_make_page_cat(titre, stats_equipe, stats_joueurs)`

Crée une page proposant le choix Équipe / Joueurs. Chaque choix mène vers une page `_make_page_stats` avec la liste de stats correspondante.

Méthode `_make_page_stats(titre, stats)`

Crée une page listant toutes les statistiques disponibles pour un sport ou une catégorie. Chaque option est un descripteur de stat issu de `_make_sports_config()`.

Méthode `_make_page_stat(stat)`

Crée la page d'exécution d'une statistique. Si le descripteur contient un sélecteur de type `options_df_fn` (DataFrame paginé), délègue vers `_make_page_selector_paged`. Sinon, gère :

1. Le sélecteur simple (`options_fn`) — affiche une liste numérotée et attend un choix.
2. Les champs libres (`inputs`) — demande une saisie pour chaque paramètre.
3. L'exécution de la fonction d'analyse (`fn(**kwargs)`).
4. L'affichage du résultat via `_afficher_resultat`.
5. Le transfert vers `_attendre_apres_resultat`.

Méthode `_make_page_selector_paged(sel, stat)`

Crée une page de sélection paginée (20 éléments par page) avec filtres. C'est la page la plus complexe de l'application. Elle maintient un dictionnaire d'état local `state` persistant entre les appels :

- `page` — index de la page courante.
- `filters` — filtres actifs sur le DataFrame.
- `df` — DataFrame chargé une seule fois.
- `_filter_applied` — flag de retour automatique après application d'un filtre.

La sous-fonction `_apply_filters(df)` applique les filtres de trois types :

- `search` (`__search__`) — recherche par préfixe sur chaque mot du nom affiché.
- `category` — égalité exacte sur une colonne.
- `year` (`__year__<col>`) — filtre par année exacte ou plage d'années (ex : 1990-2000), via `_extract_year`.

Les commandes disponibles sont `[<]`/`[>]` pour la pagination, `[f]` pour ouvrir le menu de filtres, `[r]` pour réinitialiser les filtres, et un numéro pour sélectionner un élément.

Méthode `_make_page_filter(state, filters_cfg, df)`

Crée la page listant les filtres disponibles pour un sélecteur paginé. Gère les trois types de filtres :

- **search** — saisie libre de lettres initiales.
- **category** — délègue vers `_make_page_filter_category`.
- **year** — saisie d'une année ou d'une plage (1990 ou 1990-2000). Valide le format et lève un message d'erreur si invalide.

Après application d'un filtre, le flag `_filter_applied` est levé, ce qui déclenche un retour automatique vers le sélecteur au prochain cycle.

Méthode `_make_page_filter_category(state, fconf, df)`

Crée une page paginée (20 valeurs par page) pour sélectionner une valeur de filtre catégoriel. Calcule les valeurs uniques de la colonne filtrée, les trie et les pagine. La sélection d'une valeur met à jour `state["filters"]`, lève le flag `_filter_applied` et retourne "back" pour remonter automatiquement vers le sélecteur.

Méthodes de fin de parcours de la classe CLI

Méthode `_attendre_apres_resultat(result, stat_label)`

Boucle d'attente affichée après l'exécution d'une statistique. Propose :

- `[e]` Exporter CSV — uniquement si `result` est un `DataFrame` non vide.
- `[+]` Suivant — uniquement si la pile `_fwd` n'est pas vide.
- `[-]` Retour et `[q]` Quitter — toujours présents.

La méthode boucle indéfiniment jusqu'à ce que l'utilisateur choisisse une action de navigation.

Méthode `_exporter_csv(df, stat_label)`

Sauvegarde un `DataFrame` dans un fichier CSV. Génère un nom de fichier par défaut en combinant un slug du label de la stat (caractères alphanumériques uniquement via regex `[^a-z0-9]+`) et un horodatage au format `YYYYMMDD_HHMMSS`. L'utilisateur peut accepter ce nom ou en saisir un autre. L'encodage utilisé est `utf-8-sig` (compatible avec Excel).

Fonction `main()`

Point d'entrée de l'application. Instancie la classe `CLI` et appelle `run()`. Cette fonction est appelée depuis le bloc `if __name__ == "__main__"`, ce qui permet aussi d'importer le module sans lancer l'interface.

Synthèse du flux de navigation

Page / Factory	Rôle
<code>_page_accueil</code>	Menu principal avec liste des sports filtrés
<code>_page_filtres</code>	Configuration des filtres Sport/Format/Compétition
<code>_make_page_sport</code>	Aiguillage selon le type de sport
<code>_make_page_genre</code>	Sélection Hommes / Femmes
<code>_make_page_cat</code>	Sélection Équipe / Joueurs
<code>_make_page_stats</code>	Liste des statistiques disponibles
<code>_make_page_stat</code>	Exécution d'une statistique (sélecteur simple)
<code>_make_page_selector_page</code>	Sélection paginée avec filtres (joueurs)
<code>_make_page_filter</code>	Menu des filtres disponibles
<code>_make_page_filter_category</code>	Sélection paginée d'une valeur catégorielle
<code>_attendre_apres_resultat</code>	Navigation et export après affichage d'un résultat
Valeur retournée	Effet dans la boucle <code>run()</code>
<code>callable</code>	Avancer vers la nouvelle page, empiler la courante dans <code>_back</code>
<code>"back"</code>	Reculer d'une page (dépiler <code>_back</code> , empiler dans <code>_fwd</code>)
<code>"forward"</code>	Avancer d'une page (dépiler <code>_fwd</code> , empiler dans <code>_back</code>)
<code>"quit"</code>	Terminer l'application
<code>None</code>	Rester sur la page courante (réaffichage)